One-Line Answers — the recurring prose questions

Every item below was actually asked on the 2024 or 2025 paper. Answers are sized to the point value: **1 pt ≈ one sentence, 2 pts ≈ two facts, 3–4 pts ≈ a definition + an example or a contrast.**

Do not write essays. The graders want the keyword.

Q2 — Affective Computing

Emotion theories (asked both years, 4 pts in 2025 — know the *order* of events):

- **James-Lange:** stimulus → **physiological response** → emotion. Sequential; the body reacts first, and you infer the emotion from it.
- **Cannon-Bard:** stimulus → physiological response **and** emotion **simultaneously**, independently.
- **Schachter-Singer (two-factor):** stimulus → arousal → **cognitive interpretation** of the context → emotion. Same arousal can be labelled differently depending on situation.

Dimensional model (VAD) — describe any state along three axes:

- **Valence** positive/negative, **Arousal** high/low, **Dominance** high/low (feeling in control vs controlled).
- *"Anxious, tense, helpless"* = negative valence, high arousal, **low dominance** (2025 answer).

Action Units (FACS): the smallest visually distinguishable facial muscle movements; combinations of AUs encode expressions (e.g. AU45 = blink).

Fidgeting: amount of body movement per unit time, computed from frame-to-frame pixel/landmark differences (motion energy as a % of the frame).

Mouth Aspect Ratio: ratio of vertical mouth opening to horizontal mouth width, from facial landmarks.

HRV: variability of the time between consecutive heartbeats (R-R intervals) — reflects autonomic nervous system activity. Quantified as the standard deviation $\sigma = \sqrt{\frac{1}{N}\sum_i (x_i - \bar{x})^2}$; **higher** σ = relaxed, **lower** = stress.

Skin conductance: sweat-gland activity changing skin conductivity — a direct index of **arousal** (not valence).

- **Phasic** = fast, event-related peaks. **Tonic** = slow baseline drift. Separated by **convex optimization** (cvxEDA).
- SC response features: amplitude, latency, rise time, half-recovery time.

Why artifact detection matters: physiological signals (ECG, EEG) are contaminated by muscle movement and environmental noise; uncorrected artifacts corrupt downstream features like HRV.

NOTES

Q3 — Deep Learning

Why RNNs suit sequential data (1 pt): they maintain a hidden state $\mathbf{h}_t = \sigma(U\mathbf{h}_{t-1} + W\mathbf{x}_t)$ passed from step to step, so each output depends on all previous inputs, and the same weights (U, W, V) are shared across time steps (works for variable-length input).

Vanishing gradients + LSTM (2 pts): gradients shrink multiplicatively when backpropagated through many time steps, so long-range dependencies are not learned. **LSTM** adds a **cell state** plus **forget / input / output gates**; the cell state carries information across time with little attenuation, and the gates control what is erased, written, and exposed.

Positional encoding, why: self-attention is permutation-invariant — without it the model cannot tell token order.

Skip connections: let gradients bypass layers, enabling deep stacks.

Layer norm: stabilizes training.

Masked attention (decoder): prevents attending to future tokens, preserving autoregressive generation.

Backprop trap (2025 MC): gradients **do** depend on the activation functions — the chain rule multiplies their derivatives $\sigma'(z)$ at every layer.

NOTES

Q4 — Speech Recognition

Why short-time frames for DFT (2 pts): speech is **non-stationary** — its frequency content changes as phonemes change. A DFT over the whole signal would average all of it together, so we use 20–40 ms frames (Hamming window) in which the signal is approximately stationary.

Power spectrum meaning: $P(k) = |X(k)|^2/N$ — the energy present in each frequency bin of a frame.

WER min/max (2 pts): minimum 0 % (perfect transcription). Maximum is **unbounded, above** 100 %, because insertions are not limited by the reference length.

Mel scale: perceptual, **logarithmic** frequency scale — human pitch resolution is finer at low frequencies:

$$m = 1127 \ln\left(1 + \frac{f}{700}\right)$$

Phone vs diphone: a **phone** is a single speech sound; a **diphone** runs from the middle of one phone to the middle of the next, so it **captures the co-articulation transition** between sounds — the transitions are the hard part, and cutting mid-phone puts the join in a stable region.

Architectures:

- **CTC** — aligns without frame-level labels using a **blank** token.
- **RNN-T** — encoder + prediction network + joint network; **supports streaming**.
- **Whisper** — large encoder-decoder trained on weakly-supervised multilingual data.

SpecAugment: time warping, frequency masking, time masking.

NOTES

Q5 — Large Language Models

Prompt engineering vs fine-tuning (3 pts):

- **Prompt engineering:** steer a frozen model through the input text — no weight updates, instant, cheap, but limited by context length and less reliable.
- **Fine-tuning:** update model weights on task data — stronger, more consistent, but needs labelled data, compute, and a stored copy per task.
- **Mechanism difference:** fine-tuning changes the **parameters**; prompting only changes the **input**.

Disadvantages of fine-tuning (3 pts): expensive compute/data; risk of overfitting and **catastrophic forgetting**; a separate model copy per task; must be redone when the base model updates.

top-k vs top-p (2 pts): top-k always keeps a **fixed number** k of the highest-probability tokens; top-p keeps the smallest set $V^{(p)}$ with $\sum_{t \in V^{(p)}} P(t) \geq p$, so the candidate set **adapts** to how peaked or flat the distribution is.

PEFT (LoRA, adapters): freeze the bulk of the pretrained weights, train only a small number of added parameters — cheaper and storable per task.

Quantization: lower-precision weights → smaller and faster, usually at some accuracy cost.

Pruning: remove the **least** important weights.

RAG: retrieve relevant documents at inference and condition generation on them — adds fresh/private knowledge without retraining.

Prompt strategies: zero-shot chain-of-thought ("think step by step"), few-shot examples, self-ask.

Perplexity: $\mathrm{PPL} = \left(\prod_i p_i\right)^{-1/N}$ — how surprised the model is by the text; **lower is better**.

NOTES

Q6 — Speech Synthesis

TTS pipeline: text normalization → phonetic analysis (grapheme-to-phoneme) → prosodic analysis → acoustic model → vocoder.

Homograph disambiguation (2 pts): same spelling, different pronunciation — *"Do you **live** (/l ih v/) near a zoo with **live** (/l ay v/) animals?"* Resolved by checking a homograph dictionary and using **part-of-speech tagging** to pick the right pronunciation.

Prosody — two phenomena: stress/accent (which syllable or word is prominent; a **nuclear accent** is the last and most prominent accent in a phrase) and **intonation** (the F_0 contour, e.g. rising for a question), plus phrasing/pauses and duration.

- 2025 MC: **stress depends on sentence context, accent is fixed within the word.**

Unit selection: for each target diphone s_t, pick the database unit u_j minimizing the sum of two weighted costs:

$$T(s_t, u_j) = \sum_{n=1}^{N} w_n \, T_n(s_t, u_j) \qquad J(u_j, u_{j+1}) = \sum_{p=1}^{P} w_p \, J_p(u_j, u_{j+1})$$

The **target cost** T measures how closely the unit matches the desired description (F_0, stress, duration, position); the **join cost** J measures how smoothly it joins the next unit (F_0, energy, spectral features). Crucially $J(u_j, u_{j+1}) = 0$ when the two units were **consecutive in the original recording**. Limitation: quality depends on database coverage; modern neural TTS removes this.

Tacotron 2 vs FastSpeech (key architectural difference + effect):

- **Tacotron 2: autoregressive** decoder with attention — generates mel frames one at a time; good quality but slow, and attention can fail (skipped/repeated words).
- **FastSpeech: non-autoregressive** with an explicit **duration predictor** — generates all frames in **parallel**, so far faster and more robust; needs duration supervision.
- 2024 MC trap: repeating encoder states (longer durations) makes speech **slower**, not faster.

Vocoders:

- **Griffin-Lim** — iteratively reconstructs **phase** from the magnitude spectrogram; fast, lowest quality.
- **WaveNet** — **causal dilated convolutions**, autoregressive; excellent quality but slow sample-by-sample inference.
- **HiFi-GAN** — GAN-based and parallel; **best choice for real-time**, high quality and fast.

Style transfer, integrated into the acoustic model (3 ways):

1. **One-hot speaker/style labels** fed to the encoder — simple, limited to labelled styles.
2. **Learned speaker embeddings** from a large speaker-labelled corpus, frozen and fed in — enables **zero-shot** synthesis for a new speaker from one utterance.
3. **Unsupervised style module** — a CNN/LSTM encodes the mel spectrogram of a reference clip into a style embedding, discovering styles without labels.

NOTES

Q7 — Knowledge Graphs

Format (2 pts): RDF triples — (subject, predicate, object). Applications: search, recommendation, question answering over structured facts.

Query types: one-hop (single relation), **path** (chained relations), **conjunctive** (several conditions AND-ed on the same variable).

Query2Box (conjunctive queries in embedding space) — Exercise 5, could appear as a prose question: represent each anchor entity as a **zero-volume box** (a point); apply a **projection** operator $P(\text{box}, r)$ per relation hop, which **expands** the box to cover all entities reachable by r; then apply the **intersection** operator $I(\text{box}_1, \dots, \text{box}_n)$ for the AND — its centre is a weighted sum of the input centres and its extent is the geometric intersection. Entities falling **inside the final box** are the answers.

Massive and incomplete graph → knowledge graph embeddings (e.g. TransE): SPARQL only returns facts explicitly stored, whereas embeddings generalize and support **link prediction** of missing edges.

TransE: a true triple should satisfy $\mathbf{h} + \mathbf{r} \approx \mathbf{t}$, scored by

$$\text{score}(h, r, t) = \|\mathbf{h} + \mathbf{r} - \mathbf{t}\| \qquad \text{lower is better}$$

Why TransE fails on symmetric relations: if (h, r, t) and (t, r, h) are both true then $\mathbf{h}+\mathbf{r} \approx \mathbf{t}$ **and** $\mathbf{t}+\mathbf{r} \approx \mathbf{h}$. Adding the two gives $2\mathbf{r} \approx \mathbf{0}$, so $\mathbf{r} \approx \mathbf{0}$, which forces $\mathbf{h} \approx \mathbf{t}$ — the entities collapse to the same embedding and become indistinguishable.

NOTES

Q8 — Reinforcement Learning

Formulating a task as RL (4 pts — expect a word problem):

- **State s_t:** what the agent observes now (e.g. position + velocity, or agent position + goal position).
- **Action a_t:** what it can do (e.g. movement direction/velocity, joint torques).
- **Policy $\pi(a \mid s)$:** the mapping from state to action, here an MLP; for a **stochastic** policy the network outputs a **distribution** over actions (e.g. the mean and variance of a Gaussian, or a softmax over discrete actions) that you sample from.
- **Reward r_t:** scalar feedback (e.g. progress toward goal minus a control/collision penalty).

Why RL rather than supervised learning: the environment/simulator is **not differentiable**, so you cannot backpropagate through it — and there is no ground-truth "correct action" label. The reward therefore does **not** need to be differentiable.

On-policy vs off-policy: on-policy methods (policy gradient, A2C, PPO) must use data from the **current** policy, so old trajectories cannot be reused; off-policy methods (Q-learning, DDPG, SAC) can reuse a replay buffer — more sample-efficient.

Actor-critic (A2C): the **critic** learns a value function used as a **baseline**, reducing the variance of the policy-gradient estimate; the **actor** is the policy.

PPO: a policy-optimization method that limits how far the policy changes per update, giving more stable training.

Q-learning: the policy is **implicit** — take $a^* = \arg\max_a Q(s, a)$. *(Slides mark this "for self-study" — low priority.)*

NOTES

Q9 — Animation

FK vs IK: forward kinematics maps **joint angles → end-effector position**, $\mathbf{e} = F(\theta)$; inverse kinematics finds the **joint angles that reach a target** position, $\theta = F^{-1}(\mathbf{e}^*)$.

Linear Blend Skinning: each vertex v_i is influenced by **several** bones — its new position is the weighted average of what each bone would do on its own, $v_i' = \sum_{j=1}^{B} w_{ij}\,(R_j v_i + T_j)$, with weights summing to 1. Fails at **large joint angles or twisting bone motion**, where the slides call the artifact the **"bow tie" effect** (volume reduction) — the same thing that is elsewhere called the candy-wrapper artifact. **Use the lecture's term.**

Disney "Slow In and Slow Out": motion **accelerates and decelerates** at the extremes rather than moving linearly — linear interpolation between keyframes looks robotic and abrupt.

Facial anatomy (2024 MC): facial muscles attach to bone and connect to **skin**; contracting them deforms the skin into expressions.

Manifold learning (2 pts): human motion is highly constrained, so it lies on a low-dimensional manifold. Manifold learning finds the small set of intrinsic variables (joint angles, speed) that explain the data, embedding a d-dimensional data manifold into a latent one via

$$f : \mathbb{R}^d \to \mathbb{R}^m, \qquad m < d \text{ (dimensionality reduction)}$$

making synthesis and interpolation tractable and dodging the curse of dimensionality. *Application: motion synthesis / interpolation between motions.*

DeepPhase (1 pt): a **periodic autoencoder** that learns a **motion phase manifold** — extracting periodic phase variables from motion curves so that cyclic motion like locomotion aligns properly and blends without foot sliding.

Normalizing flows (2 pts): describe a complex distribution as a chain of **simple, invertible, differentiable** transformations of a simple prior $z \sim \mathcal{N}(0, 1)$:

$$x = f(z) = f_1(f_2(\dots f_N(z))), \qquad z = f_N^{-1}(\dots f_1^{-1}(x))$$

Because each step is invertible you can both **sample** and **evaluate the exact density** $p(x)$ via the change-of-variables formula (using the Jacobian determinant) — combining the benefits of generative models and explicit PDF modelling. *Application: generating diverse character motion.*

Autonomous agents: operate independently, perceive and react to their environment, and act **goal-directedly over time** — not solely from immediate perception (2025 MC).

Dialogue trees: predictable and controllable, but **low flexibility** — they cannot handle unexpected queries and grow unmanageable as branches multiply.

NOTES